Heimalarm

KONZEPT · EIGENENTWICKLUNG · OHNE CLOUD

Alarm → PC & Smart Home

Ein ESP32 liest den Alarm direkt an der Ladeschale. Eine kleine Zentrale im Heimnetz schaltet Licht und Geräte, und am PC fragt die eigene App: **„Fährst du?“** Kostenlos, Open Source, ohne Cloud.

Stand	Melder	Laufende Kosten
17.10.2026	Motorola TPG2200	keine

DAS SYSTEM AUF EINEN BLICK

AUSLÖSER

Melder in Ladeschale
Relais schließt bei Alarm

ESP32
liest den Kontakt, sendet sofort

aPager / SYBOS
Webhook oder Benachrichtigung

ZENTRALE (RASPBERRY PI)

Mosquitto
MQTT-Server, Nachrichtendrehscheibe

Regel-Dienst
eigene Routinen, läuft immer

Zigbee2MQTT
+ USB-Stick: Lampen, Sensoren

AKTION

Licht & Geräte
An · 1 % · Küche an

App Handy
Anlernen · Steuern · Routinen

App Windows 11
„Fährst du?“ → Ruhezustand

Kostenlos & offen
Keine Abos, keine Cloud. Nur einmalige Hardware und Open-Source-Software.

Lokal
Alles läuft im Heimnetz. Kein Internet nötig, keine Verzögerung durch eine Cloud.

Nur Komfort
Die Automation unterstützt. Die eigentliche Alarmierung bleibt der Melder.

01 Ausgangslage
02 ESP32 an der Ladeschale
03 Bauanleitung ESP32
04 Zentrale & App
05 Licht · Herunterfahren
06 Funktionsliste · Kosten

01 AUSGANGSLAGE

Gleicher Melder, gleicher Kontakt

Beide nutzen den Motorola TPG2200. Weil die Ladeschale einen Relaisausgang hat, kann jeder von euch den Alarm auf **dieselbe Weise** abgreifen, unabhängig von der Alarm-App.

ICH

Melder	Motorola TPG2200
Alarm-App	SYBOS keine Webhooks
Heute	Funkhandel-Alarmschalter (bleibt als 2. Weg)
Neu	ESP32 am Relaisausgang

KOLLEGE

Melder	Motorola TPG2200
Alarm-App	aPager PRO Webhooks
Heute	Webhook (Smart-Home-Abo nötig)
Neu	ESP32 am Relaisausgang, Webhook optional

ZWEITER WEG

Der Funkhandel-Schalter bleibt, wie er ist

Der Alarmschalter läuft weiter unverändert mit ELESION und Alexa: Licht an, Ansage. Er gehört **nicht** zum neuen System und hat keine Verbindung zur eigenen App.

Vorteil: Zwei unabhängige Wege. Fällt die Zentrale aus, geht das Licht trotzdem an, und umgekehrt.

02 ALARMSIGNAL

ESP32 direkt an der Ladeschale

Die Homestation des TPG2200 (PMPN4142) hat ein Relais, das bei Alarm schließt. In dieser 2-poligen Buchse steckt auch der Funkhandel-Schalter, beide teilen sich den Kontakt. Ein ESP32 für ca. 5–10 € liest den Kontakt direkt.

Verdrahtung

LADESCHALE
Relaisausgang 2-polig
schließt bei Alarm

Kontakt A → GPIO

Kontakt B → GND

ESP32
Eingang mit Pull-up
offen = HIGH · Alarm = LOW
eigene Firmware (Arduino) oder ESPHome

→

WLAN · LOKAL
MQTT an Zentrale
optional zusätzlich HTTP direkt an den PC

Relais laut Handbuch

Schaltstrom	2 A bei 30 V DC · 0,5 A bei 125 V AC
max. Spannung	220 V DC · 125 V AC
max. Leistung	60 W · 62,5 VA

Für die 3,3-V-Abfrage eines ESP32 mehr als ausreichend.

Vorher messen. Das Handbuch sagt nicht, ob der Kontakt potentialfrei ist. Mit dem Multimeter prüfen: Spannung zwischen den Polen (mit und ohne Alarm) sollte 0 V sein, Widerstand offen ∞, geschlossen ≈ 0 Ω. Liegt Spannung an: Optokoppler dazwischen.

Was die Firmware tut

1 · ERKENNEN
Kontakt schließt
Entprellung ~50 ms

→

2 · MELDEN
MQTT-Nachricht
`alarm/ich`

→

3 · ABSICHERN
Wiederholen
bis Zentrale bestätigt

→

4 · LEBENSZEICHEN
„online"-Status
App zeigt Ausfall an

Strom per USB-Netzteil. Der Kollege baut denselben ESP32 an seine Ladeschale und braucht damit kein Smart-Home-Abo mehr.

03 BAUANLEITUNG

ESP32 an die Ladeschale, Schritt für Schritt

> Der Funkhandel-Schalter bleibt stecken. ESP32 und Schalter teilen sich den Relaiskontakt über einen 2-Pol-Y-Verteiler.
> **Material:** ESP32-DevKit · USB-Netzteil + Kabel · 2-Pol-Y-Verteiler bzw. 2-Pol-Stecker · 2 Jumper-Kabel · Multimeter · Chrome/Edge am PC

1. **Messen.** Spannung zwischen den Polen mit und ohne Alarm: 0 V? Widerstand: offen ∞, Alarm ≈ 0 Ω. Liegt Spannung an → Optokoppler.
2. **Verdrahten.** Pol A → `GPIO27`, Pol B → `GND`. Kein Widerstand nötig, der interne Pull-up ist aktiv. Nicht GPIO34–39 nehmen (dort gibt es keinen Pull-up).
3. **Flashen.** ESP32 per USB an den PC, `web.esphome.io` öffnen, „Prepare for first use“, danach die Konfiguration unten einspielen.
4. **Testen.** Erst ESP32 allein, dann mit Schalter am Y-Verteiler: Beide müssen bei Testalarm auslösen. Tut es einer nicht → Optokoppler für den ESP32. Auf der Zentrale: `mosquitto_sub -t 'alarm/#' -v`
5. **Montieren.** ESP32 in ein kleines Gehäuse, neben die Ladeschale kleben, Netzteil dauerhaft einstecken.

Konfiguration (ESPHome)

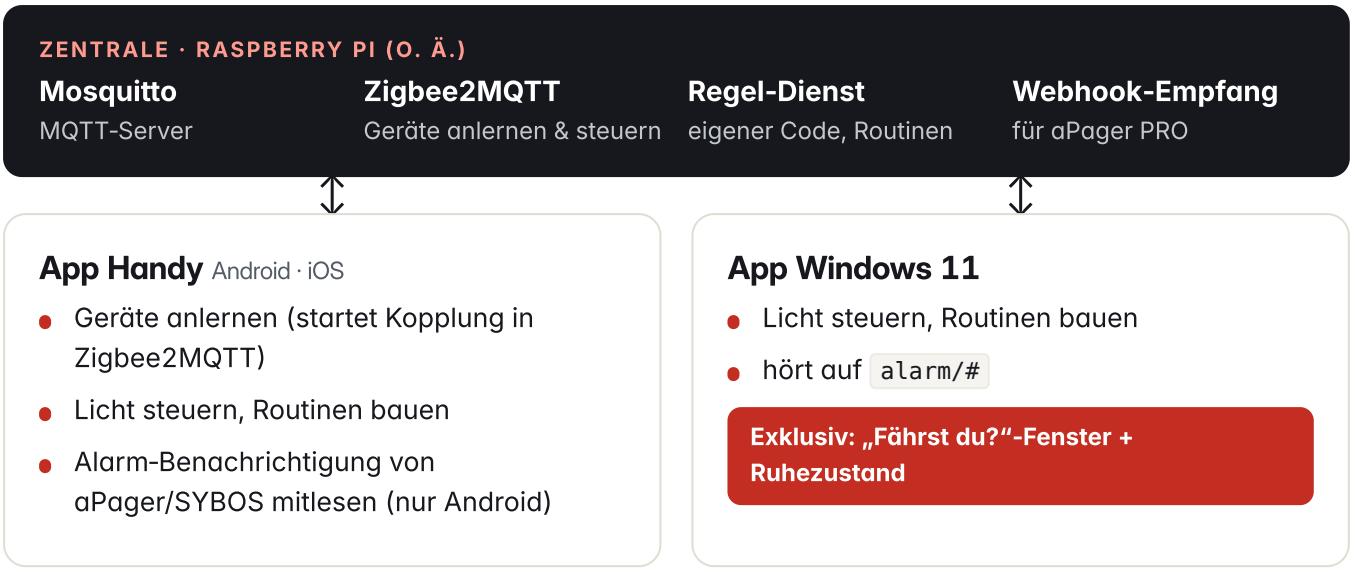
```
esphome:
  name: alarm-ich
esp32:
  board: esp32dev
wifi:
  ssid: !secret wifi_ssid
  password: !secret wifi_pw
mqtt:                       # statt api:
  broker: 192.168.1.10      # IP der Zentrale
  topic_prefix: alarm/ich   # Status: alarm/ich/status
binary_sensor:
  - platform: gpio
    name: "Melder Alarm"
    pin:
      number: GPIO27
      mode: { input: true, pullup: true }
      inverted: true        # Kontakt zu = Alarm = ON
    filters:
      - delayed_on: 50ms    # Entprellen
    on_press:
      - mqtt.publish: { topic: alarm/ich, payload: "ALARM", qos: 1 }
```

Der Kollege nimmt dieselbe Datei mit `name: alarm-kollege` und `alarm/kollege`. Die App abonniert `alarm/#` und erkennt Ausfälle über den Status `offline`.

04 ZENTRALE & APP

Eine Zentrale, eine App, drei Plattformen

Der PC geht in den Ruhezustand und das Handy ist nicht immer zuhause. Deshalb läuft die Logik auf einer kleinen Zentrale, die immer an ist. Die **Flutter-App** (Android, iOS, Windows aus einer Codebasis) verbindet sich nur per MQTT damit.



Funktionen je Plattform

FUNKTION	ANDROID	IOS	WINDOWS
Geräte anlernen, Licht und Routinen	●	●	●
Alarm vom ESP32 empfangen (MQTT)	●	●	●
aPager/SYBOS-Benachrichtigung mitlesen	●	○	—
„Fährst du?"-Fenster + Ruhezustand	—	—	●

● verfügbar ○ nicht möglich — nicht relevant

Konfiguration je Gerät: Hibernate, Countdown und Autostart gibt es nur am PC. Routinen liegen in der Zentrale und laufen dadurch auch, wenn Handy und PC aus sind.

05 LICHT & GERÄTE

Anlernen und Steuern ohne Cloud

Zigbee2MQTT ist Open Source, läuft komplett lokal, unterstützt über 5.700 Geräte und braucht kein Home Assistant. Die eigene App spricht direkt per MQTT damit.

NEU KAUFEN

Zigbee-Geräte

Lampen, Stecker, Schalter mit Zigbee. Anlernen über die eigene App, die Zigbee2MQTT in den Kopplungsmodus schaltet.

VORERST BEHALTEN

Vorhandene Lampen

Bleiben in ELESION und werden weiter vom Funkhandel-Schalter geschaltet (2. Weg). Nach und nach durch Zigbee-Lampen ersetzen.

BASTELN

Freie Firmware

WLAN-Lampen mit OpenBeken oder ESPHome flashen. Ob das geht, hängt vom verbauten Chip ab.

HERUNTERFAHREN

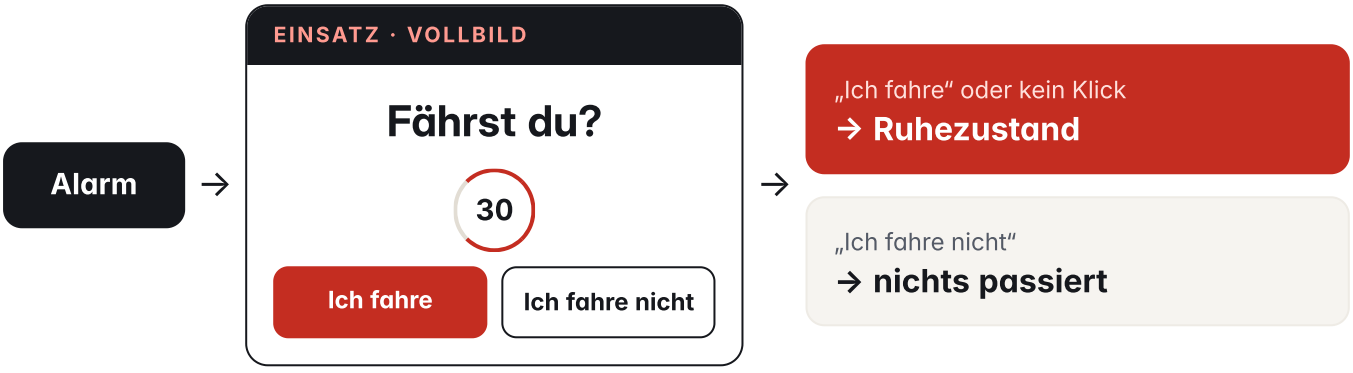
Ruhezustand statt „alles speichern“

Einen Windows-Befehl „alle Apps speichern“ gibt es nicht. Windows fragt nur, ob Programme beendet werden dürfen (`WM_QUERYENDSESSION`). Ein Programm mit ungespeicherten Daten zeigt „Speichern?“ und blockiert dann unbegrenzt. `shutdown /f` erzwingt es, mit Datenverlust.

VARIANTE	DATENVERLUST	ZUVERLÄSSIG OHNE NUTZER
Ruhezustand Empfehlung	nein	ja
Herunterfahren (sanft)	nein	hängt an Dialogen
Herunterfahren (`/f`)	ja	ja
Mischform: sanft, nach 2 min Ruhezustand	nein	ja

Einmalig als Admin: `powercfg /h on`

Ablauf am PC



06 FUNKTIONSLISTE

Einstellungen & Funktionen der App

Merkliste für die Entwicklung: Haken setzen, wenn umgesetzt. ★ = Grundausstattung für die erste Version.

A · Alarmeingang (ESP32 & Quellen)

☐	A1	Quellen ein/aus: ESP32, aPager-Webhook, Benachrichtigung mitlesen (Android)	★
☐	A2	Entprellzeit des Kontakts (z. B. 50 ms)	★
☐	A3	Mindestdauer, ab der ein Kontakt als Alarm zählt	
☐	A4	Sperrzeit nach einem Alarm (z. B. 60 s)	★
☐	A5	Doppelte Meldungen mehrerer Quellen zu einem Alarm zusammenführen	
☐	A6	Filter für Benachrichtigungen: App, Schlüsselwörter	
☐	A7	Stichwort und Einheit aus dem aPager-Webhook übernehmen	
☐	A8	Warnung, wenn der ESP32 offline ist, mit Wartezeit	★
☐	A9	Lebenszeichen-Intervall des ESP32	
☐	A10	Firmware-Update des ESP32 über WLAN aus der App	
☐	A11	Testmodus: Kontakt simulieren ohne echte Aktion	

B · Routinen & Regeln

☐	B1	Mehrere Routinen mit Name, aktiv/inaktiv	★
☐	B2	Auslöser: Alarm, Uhrzeit, Gerätezustand, manueller Knopf	★
☐	B3	Bedingungen: Uhrzeit/Nachtmodus, Wochentag, Sonnenzeiten	★
☐	B4	Bedingung „Wer ist zuhause“ (Handy im Heim-WLAN)	
☐	B5	Bedingung nach Stichwort oder Einheit	
☐	B6	Aktionen in Reihenfolge, mit Verzögerung	★
☐	B7	Zeitlich begrenzte Aktionen (nach X min wieder aus)	★
☐	B8	Rückkehr-Routine stellt vorherigen Zustand her	
☐	B9	Urlaubs-/Abwesenheitsmodus pausiert Routinen	
☐	B10	Bedingungen per UND/ODER verknüpfen	
☐	B11	Routinen exportieren/importieren	

C · Licht & Geräte (Zigbee2MQTT)

☐	C1	Geräte anlernen (mit Zeitlimit), umbenennen, entfernen	★
☐	C2	Räume und Gruppen	★
☐	C3	Helligkeit, Farbe, Farbtemperatur je Gerät/Gruppe	★
☐	C4	Übergangszeit (sanft hochdimmen)	
☐	C5	Alarmfarbe bzw. Alarmszene	
☐	C6	Blinken/Effekt bei Alarm, wenn unterstützt	
☐	C7	Direktverknüpfung Schalter–Lampe (Binding)	
☐	C8	Erreichbarkeit jedes Geräts	
☐	C9	Geräte-Firmware prüfen, einspielen, planen	
☐	C10	Netzwerkkarte des Zigbee-Netzes	
☐	C11	Steckdosen und Schalter einbinden	

D · Windows-PC

☐	D1	Aktion: Ruhezustand, Energiesparen, Sperren, Herunterfahren, Mischform, nichts	★
☐	D2	Länge des Countdowns	★
☐	D3	Standard bei keinem Klick: ausführen oder abbrechen	★
☐	D4	Texte und Knöpfe des Fensters anpassen	
☐	D5	Ton und Lautstärke, auch bei stummem PC	
☐	D6	Fenster über allem, auch über Vollbild	
☐	D7	Ausnahmen, z. B. bestimmtes Programm läuft	
☐	D8	Vorher Medien pausieren / Programme anstoßen	
☐	D9	Autostart, Symbol im Infobereich	★
☐	D10	Ruhezustand mit einem Klick prüfen/einrichten	
☐	D11	Wake-on-LAN: PC über die App aufwecken	
☐	D12	Zeitfenster ohne Reaktion	

E · Handy-App

- ☐ E1 Alarm-Benachrichtigung an/aus, Ton ★
- ☐ E2 Widget mit Schnellknöpfen (Licht, Testalarm)
- ☐ E3 Dashboard: Status ESP32, Zentrale, PC, Geräte
- ☐ E4 Alarmverlauf mit Quelle, Uhrzeit, Aktionen
- ☐ E5 Hell/Dunkel, Schriftgröße, Sprache
- ☐ E6 Android: Akku-Optimierung prüfen und Hinweis
- ☐ E7 Protokoll aller Ereignisse: wann was passiert, gemacht oder ausgeführt wurde, filterbar ★

F · Zentrale & System

- ☐ F1 Adresse der Zentrale (Auto-Suche oder manuell) ★
- ☐ F2 MQTT-Zugangsdaten ★
- ☐ F3 Backup und Wiederherstellung der Konfiguration
- ☐ F4 Updates der Zentrale anzeigen
- ☐ F5 Logs der Zentrale einsehen
- ☐ F6 Zeitzone, Standort (Sonnenzeiten)

G · Personen & Mehrbenutzer

- ☐ G1 Mehrere Personen/Melder (je eigener ESP32)
- ☐ G2 Routinen pro Person
- ☐ G3 Rollen: Admin und Nutzer

H · Sicherheit

- ☐ H1 Verschlüsselte Verbindung (TLS) ★
- ☐ H2 Geheimer Schlüssel für den Webhook
- ☐ H3 Nur Heimnetz, optional VPN von unterwegs
- ☐ H4 App-Sperre per PIN/Fingerabdruck

I · Test & Diagnose

- ☐ I1 Testalarm: Routine ja, PC schläft nicht ★
- ☐ I2 Komplettprobe mit allen Aktionen
- ☐ I3 Verbindungstest Zentrale, ESP32, PC, Geräte
- ☐ I4 Wöchentlicher Selbsttest mit Bericht

07 KOSTEN

Nur einmalige Hardware

TEIL	ZWECK	CA. KOSTEN
ESP32 + USB-Netzteil, 2-Pol-Stecker	Alarm an der Ladeschale (je Person einer)	5–10 €
Zigbee-USB-Stick	Zigbee2MQTT, Geräte anlernen	ca. 20 €
Raspberry Pi	Zentrale (entfällt, wenn schon vorhanden)	je nach Modell
Mosquitto, Zigbee2MQTT, Flutter, ESPHome/Arduino	Software	kostenlos

08 FAHRPLAN

In fünf Stufen zum Ziel

1. **ESP32 an der Ladeschale** + Mosquitto auf der Zentrale — Alarm lokal verfügbar
2. **Windows-App:** „Fährst du?"-Fenster, Ruhezustand — PC reagiert
3. **Zigbee2MQTT** + Regel-Dienst — Licht bei Alarm
4. **Handy-App:** Anlernen, Steuern, Routinen — eigene Steuerung
5. **Extras:** aPager-Webhook, Benachrichtigungen, iOS — optional

ANHANG

Quellen

THEMA	QUELLE
Ladeschale	Motorola TPG2200 – Relaisspezifikation (Manualslib) Raspberry-Pi-Forum – TPG2200-Relais abfragen
Alarmschalter	Funkhandel – Alarmschalter Smart Home TPG2200
ESPHome	ESPHome – GPIO Binary Sensor ESPHome – MQTT Client
Zigbee	raspberry.tips – Zigbee2MQTT einrichten
aPager PRO	Alamos – Webhook-Format Alamos – Benachrichtigung als Auslöser
Windows	Microsoft – Shutting Down Microsoft – Application Shutdown Changes